

Отчёт по лабораторной работе №4

Моделирование сетей передачи данных

Ищенко Ирина Олеговна

Содержание

1	Цель работы	4
2	Задание	5
3	Выполнение лабораторной работы	6
3.1	Добавление/изменение задержки в эмулируемой глобальной сети	8
3.2	Изменение задержки в эмулируемой глобальной сети	10
3.3	Восстановление исходных значений (удаление правил) задержки в эмулируемой глобальной сети	11
3.4	Добавление значения дрожания задержки в интерфейс подключения к эмулируемой глобальной сети	12
3.5	Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети	13
3.6	Распределение задержки в интерфейсе подключения к эмулируемой глобальной сети	14
3.7	Воспроизведение экспериментов. Добавление задержки для интерфейса, подключающегося к эмулируемой глобальной сети .	15
3.8	Задание для самостоятельной работы	20
4	Выводы	25

Список иллюстраций

3.1	Простейшая топология	6
3.2	ifconfig на хостах h1 и h2	7
3.3	Проверка подключения между хостами	8
3.4	Добавление задержки в 100мс	9
3.5	Двунаправленная задержка соединения	10
3.6	Изменение задержки на 50мс	11
3.7	Восстановление исходных значений задержки	12
3.8	Добавление значения дрожания задержки в интерфейс подключения	13
3.9	Добавление значения корреляции для джиттера и задержки в интерфейс подключения	14
3.10	Распределение задержки в интерфейсе подключения	15
3.11	Скрипт	16
3.12	скрипт для визуализации ping_plot	16
3.13	Makefile для управления процессом проведения эксперимента . .	17
3.14	результат выполнения скриптов	17
3.15	результат выполнения скриптова	18
3.16	Скрипт rtt.py	18
3.17	Makefile	19
3.18	Результат работы скрипта rtt.py	19
3.19	Просмотр графика ping.png	20
3.20	Просмотр минимального, среднего, максимального и стандартного отклонения времени приёма-передачи	21
3.21	Просмотр графика ping.png	22
3.22	Просмотр минимального, среднего, максимального и стандартного отклонения времени приёма-передачи	22
3.23	Просмотр графика ping.png	23
3.24	Просмотр минимального, среднего, максимального и стандартного отклонения времени приёма-передачи	23
3.25	Просмотр графика ping.png	24
3.26	Просмотр минимального, среднего, максимального и стандартного отклонения времени приёма-передачи	24

1 Цель работы

Основной целью работы является знакомство с NETEM — инструментом для тестирования производительности приложений в виртуальной сети, а также получение навыков проведения интерактивного и воспроизводимого экспериментов по измерению задержки и её дрожания (jitter) в моделируемой сети в среде Mininet.

2 Задание

1. Задайте простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8.
2. Проведите интерактивные эксперименты по добавлению/изменению задержки, джиттера, значения корреляции для джиттера и задержки, распределения времени задержки в эмулируемой глобальной сети.
3. Реализуйте воспроизводимый эксперимент по заданию значения задержки в эмулируемой глобальной сети. Постройте график.
4. Самостоятельно реализуйте воспроизводимые эксперименты по изменению задержки, джиттера, значения корреляции для джиттера и задержки, распределения времени задержки в эмулируемой глобальной сети. Постройте графики.

3 Выполнение лабораторной работы

Запустим виртуальную среду с mininet. Из основной ОС подключимся к виртуальной машине. В виртуальной машине mininet при необходимости исправим права запуска X-соединения. Скопируем значение куки (MIT magic cookie) своего пользователя mininet в файл для пользователя root.

Зададим простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8 (Рисунок 3.1).

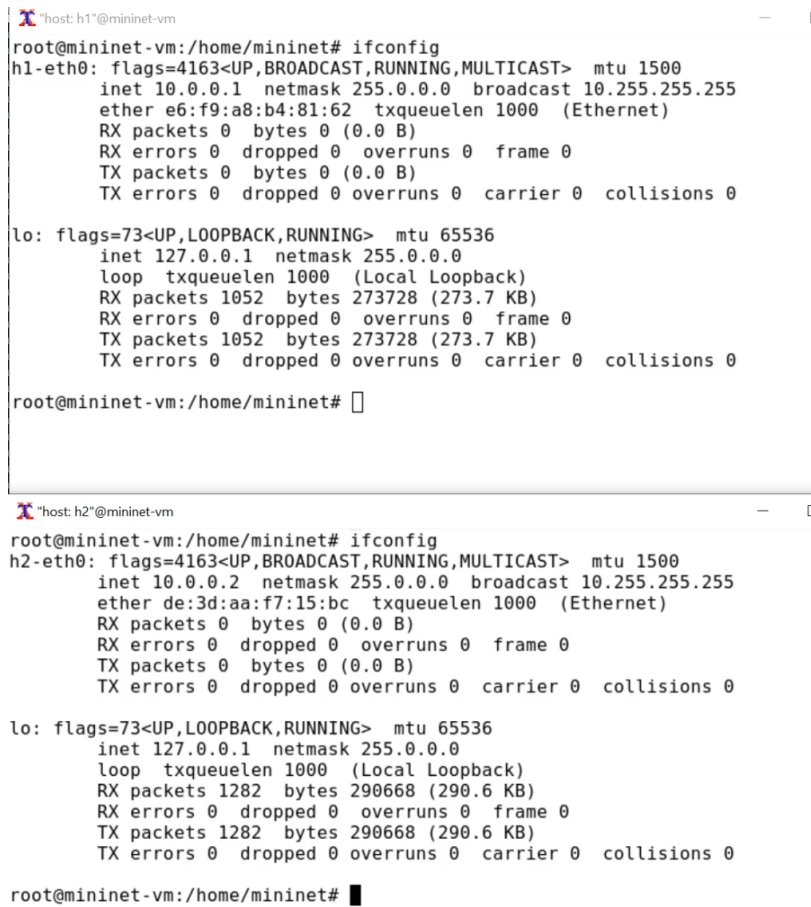
После введения этой команды запустятся терминалы двух хостов, коммутатора и контроллера. Терминалы коммутатора и контроллера можно закрыть.

```
mininet@mininet-vm:~$ sudo mn --topo=single,2 -x
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Running terms on localhost:10.0
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

Рисунок 3.1: Простейшая топология

На хостах h1 и h2 введем команду ifconfig, чтобы отобразить информацию, относящуюся к их сетевым интерфейсам и назначенным им IP-адресам.

В дальнейшем при работе с NETEM и командой `tc` будут использоваться интерфейсы `h1-eth0` и `h2-eth0` (Рисунок 3.2).



```
root@mininet-vm:/home/mininet# ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.0.0.0 broadcast 10.255.255.255
    ether e6:f9:a8:b4:81:62 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1052 bytes 273728 (273.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1052 bytes 273728 (273.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet#

root@mininet-vm:/home/mininet# ifconfig
h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.2 netmask 255.0.0.0 broadcast 10.255.255.255
    ether de:3d:aa:f7:15:bc txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1282 bytes 290668 (290.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1282 bytes 290668 (290.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet#
```

Рисунок 3.2: `ifconfig` на хостах `h1` и `h2`

Проверим подключение между хостами `h1` и `h2` с помощью команды `ping` с параметром `-c 6` (Рисунок 3.3).

```

root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=5.02 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.293 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.111 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.064 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.066 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.118 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5078ms
root@mininet-vm:/home/mininet#
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
loop txqueuelen 1000 (Local Loopback)
RX packets 1282 bytes 290668 (290.6 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 1282 bytes 290668 (290.6 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions

root@mininet-vm:/home/mininet# ping 10.0.0.1 -c 6
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=2.60 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.051 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.069 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.093 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.060 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.061 ms

--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5096ms
rtt min/avg/max/mdev = 0.051/0.489/2.604/0.945 ms
root@mininet-vm:/home/mininet#

```

Рисунок 3.3: Проверка подключения между хостами

3.1 Добавление/изменение задержки в эмулируемой глобальной сети

На хосте h1 добавим задержку в 100 мс к выходному интерфейсу (Рисунок 3.4).

```
sudo tc qdisc add dev h1-eth0 root netem delay 100ms
```

- sudo: выполнить команду с более высокими привилегиями;
- tc: вызвать управление трафиком Linux;
- qdisc: изменить дисциплину очередей сетевого планировщика;
- add: создать новое правило;

- dev h1-eth0: указать интерфейс, на котором будет применяться правило;
- netem: использовать эмулятор сети;
- delay 100ms: задержка ввода 100 мс.

Проверим, что соединение от хоста h1 к хосту h2 имеет задержку 100 мс, используя команду ping с параметром -c 6 с хоста h1. Минимальное - (100.404), среднее - (101.004), максимальное - (102.023), стандартное отклонение времени (0.611) приема-передачи.

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 100ms
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms
--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5010ms
rtt min/avg/max/mdev = 100.404/101.004/102.023/0.611 ms
root@mininet-vm:/home/mininet#
```

Рисунок 3.4: Добавление задержки в 100мс

Для эмуляции глобальной сети с двунаправленной задержкой необходимо к соответствующему интерфейсу на хосте h2 также добавим задержку в 100 миллисекунд (Рисунок 3.5).

Проверим, что соединение между хостом h1 и хостом h2 имеет RTT в 200 мс (100 мс от хоста h1 к хосту h2 и 100 мс от хоста h2 к хосту h1), повторив команду ping с параметром -c 6 на терминале хоста h1. Минимальное - (200.668), среднее - (201.654), максимальное - (202.485), стандартное отклонение времени (0.769) приема-передачи.

```

root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=202 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=202 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=201 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=201 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=201 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=202 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5011ms
rtt min/avg/max/mdev = 200.668/201.654/202.485/0.769 ms
root@mininet-vm:/home/mininet#

```

```

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1282 bytes 290668 (290.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1282 bytes 290668 (290.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# ping 10.0.0.1 -c 6
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=2.60 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.051 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.069 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.093 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.060 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.061 ms

--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5096ms
rtt min/avg/max/mdev = 0.051/0.489/2.604/0.945 ms
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h2-eth0 root netem delay 1
00ms
root@mininet-vm:/home/mininet#

```

Рисунок 3.5: Двухнаправленная задержка соединения

3.2 Изменение задержки в эмулируемой глобальной сети

Изменим задержку со 100 мс до 50 мс для отправителя h1 и для получателя h2 (Рисунок 3.6).

Проверим, что соединение от хоста h1 к хосту h2 имеет задержку 100 мс, используя команду `ping` с параметром `-c 6` с терминала хоста h1. Минимальное - (100.990), среднее - (101.579), максимальное - (102.512), стандартное отклонение времени (0.529) приема-передачи.

```

root@mininet-vm:/home/mininet# sudo tc qdisc change dev h1-eth0 root netem delay 50ms
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5012ms
rtt min/avg/max/mdev = 100.990/101.579/102.512/0.529 ms
root@mininet-vm:/home/mininet# ifconfig h1-eth0 netmask 255.0.0.0
loop txqueuelen 1000 (Local Loopback)
RX packets 1282 bytes 290668 (290.6 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 1282 bytes 290668 (290.6 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# ping 10.0.0.1 -c 6
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=2.60 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.051 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.069 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.093 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.060 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.061 ms

--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5096ms
rtt min/avg/max/mdev = 0.051/0.489/2.604/0.945 ms
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h2-eth0 root netem delay 100ms
root@mininet-vm:/home/mininet# sudo tc qdisc change dev h2-eth0 root netem delay 50ms
root@mininet-vm:/home/mininet#

```

Рисунок 3.6: Изменение задержки на 50мс

3.3 Восстановление исходных значений (удаление правил) задержки в эмулируемой глобальной сети

Восстановим конфигурацию по умолчанию, удалив все правила, применённые к сетевому планировщику соответствующего интерфейса для отправителя h1 и для получателя h2. Проверим, что соединение между хостом h1 и хостом h2 не имеет явно установленной задержки, используя команду ping с параметром -c 6 с терминала хоста h1 (Рисунок 3.7). Минимальное - (0.055), среднее - (0.106), максимальное - (0.161), стандартное отклонение времени (0.032) приема-передачи.

```

root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.161 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.099 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.055 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.095 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.100 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.127 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5105ms
rtt min/avg/max/mdev = 0.055/0.106/0.161/0.032 ms
root@mininet-vm:/home/mininet#
loop - txqueuelen 1000 (Local Loopback)
RX packets 1282 bytes 290668 (290.6 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 1282 bytes 290668 (290.6 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# ping 10.0.0.1 -c 6
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=2.60 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.051 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.069 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.093 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.060 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.061 ms

--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5096ms
rtt min/avg/max/mdev = 0.051/0.489/2.604/0.945 ms
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h2-eth0 root netem delay 1
00ms
root@mininet-vm:/home/mininet# sudo tc qdisc change dev h2-eth0 root netem dela
y 50ms
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h2-eth0 root netem
root@mininet-vm:/home/mininet#

```

Рисунок 3.7: Восстановление исходных значений задержки

3.4 Добавление значения дрожания задержки в интерфейс подключения к эмулируемой глобальной сети

Добавим на узле h1 задержку в 100 мс со случайным отклонением 10 мс. Проверим, что соединение от хоста h1 к хосту h2 имеет задержку 100 мс со случайным отклонением ± 10 мс, используя в терминале хоста h1 команду ping с параметром -c 6. Минимальное - (92.175), среднее - (98.089), максимальное - (103.404), стандартное отклонение времени (4.067) приема-передачи. Восстановим конфигурацию интерфейса по умолчанию на узле h1 (Рисунок 3.8).

```

root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 1
00ms 10ms
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=99.7 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=92.2 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=97.3 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=93.9 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=102 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5005ms
rtt min/avg/max/mdev = 92.175/98.089/103.404/4.067 ms
root@mininet-vm:/home/mininet# █

```

Рисунок 3.8: Добавление значения дрожания задержки в интерфейс подключения

3.5 Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети

Добавим на интерфейсе хоста h1 задержку в 100 мс с вариацией ± 10 мс и значением корреляции в 25%. Убедимся, что все пакеты, покидающие устройство h1 на интерфейсе h1-eth0, будут иметь время задержки 100 мс со случайным отклонением ± 10 мс, при этом время передачи следующего пакета зависит от предыдущего значения на 25%. Используем для этого в терминале хоста h1 команду ping с параметром -c 20. Минимальное - (92.802), среднее - (102.133), максимальное - (109.745), стандартное отклонение времени (5.578) приема-передачи. Восстановим конфигурацию интерфейса по умолчанию на узле h1 (Рисунок 3.9).

```

root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 1
00ms 10ms 25%
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 20
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=107 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=94.1 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=107 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=94.8 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=98.9 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=92.8 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=110 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=107 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=96.4 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=93.1 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=98.2 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=110 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=109 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=105 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=107 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19041ms
rtt min/avg/max/mdev = 92.802/102.133/109.745/5.578 ms
root@mininet-vm:/home/mininet# █

```

Рисунок 3.9: Добавление значения корреляции для джиттера и задержки в интерфейс подключения

3.6 Распределение задержки в интерфейсе подключения к эмулируемой глобальной сети

Зададим нормальное распределение задержки на узле h1 в эмулируемой сети. Убедимся, что все пакеты, покидающие хост h1 на интерфейсе h1-eth0, будут иметь время задержки, которое распределено в диапазоне 100 мс $\pm$ 20 мс. Используем для этого команду ping на терминале хоста h1 с параметром -с 10. Минимальное - (82.720), среднее - (105.785), максимальное - (129.487), стандартное отклонение времени (14.727) приема-передачи. Восстановим конфигурацию интерфейса по умолчанию на узле h1. Завершим работу mininet в интерактивном режиме (Рисунок 3.10).

```

root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 1
00ms 20ms distribution normal
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 10
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=82.7 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=111 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=119 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=110 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=129 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=87.5 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=124 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=93.2 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9011ms
rtt min/avg/max/mdev = 82.720/105.785/129.487/14.727 ms
root@mininet-vm:/home/mininet# █

```

Рисунок 3.10: Распределение задержки в интерфейсе подключения

3.7 Воспроизведение экспериментов. Добавление задержки для интерфейса, подключающегося к эмулируемой глобальной сети

С помощью API Mininet воспроизведем эксперимент по добавлению задержки для интерфейса хоста, подключающегося к эмулируемой глобальной сети. В виртуальной среде mininet в своём рабочем каталоге с проектами создадим каталог simple-delay и перейдем в него.

Создадим скрипт для эксперимента lab_netem_i.py (Рисунок 3.11).

```

mininet@mininet-vm: ~/work/lab_netem_i/simple-delay
#!/usr/bin/env python

"""
Simple experiment.
Output: ping.dat
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
import time

def emptyNet():
    """Create an empty network and add nodes to it."""

    net = Mininet( controller=Controller,
                    waitConnected=True )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( '*** Adding switch\n' )
    s1 = net.addSwitch( 's1' )

    info( '*** Creating links\n' )
    net.addLink( h1, s1 )
    net.addLink( h2, s1 )

    info( '*** Starting network\n' )
    net.start()

    info( '*** Set delay\n' )
    h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem delay 100ms' )
    h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem delay 100ms' )

    time.sleep(10) # Wait 10 seconds

    info( '*** Ping\n' )
    h1.cmdPrint( 'ping -c 100', h2.IP(), '| grep "time=" | awk \'{print $5, $7}\'' | se
d -e \'/s/time=//g\' -e \'/s/icmp_seq=//g\' > ping.dat' )

    info( '*** Stopping network' )
    net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()

```

Рисунок 3.11: Скрипт

Создаём скрипт для визуализации ping_plot результатов эксперимента (Рисунок 3.12).

```

mininet@mininet-vm: ~/work/lab_netem_i/simple-delay
#!/usr/bin/gnuplot --persist

set terminal png crop
set output 'ping.png'
set xlabel "Sequence number"
set ylabel "Delay (ms)"
set grid
plot "ping.dat" with lines

```

Рисунок 3.12: скрипт для визуализации ping_plot

Зададим права доступа к файлу скрипта: `chmod +x ping_plot`.

Создадим Makefile для управления процессом проведения эксперимента (Рисунок 3.13).

```
mininet@mininet-vm: ~/work/lab_netem_i/simple-delay
all: ping.dat ping.png

ping.dat:
    sudo python lab_netem_i.py
    sudo chown mininet:mininet ping.dat

ping.png: ping.dat
    ./ping_plot

clean:
    rm -f *.dat *.png
```

Рисунок 3.13: Makefile для управления процессом проведения эксперимента

Выполним эксперимент. Продемонстрируем построенный в результате выполнения скриптов график (Рисунок 3.14).

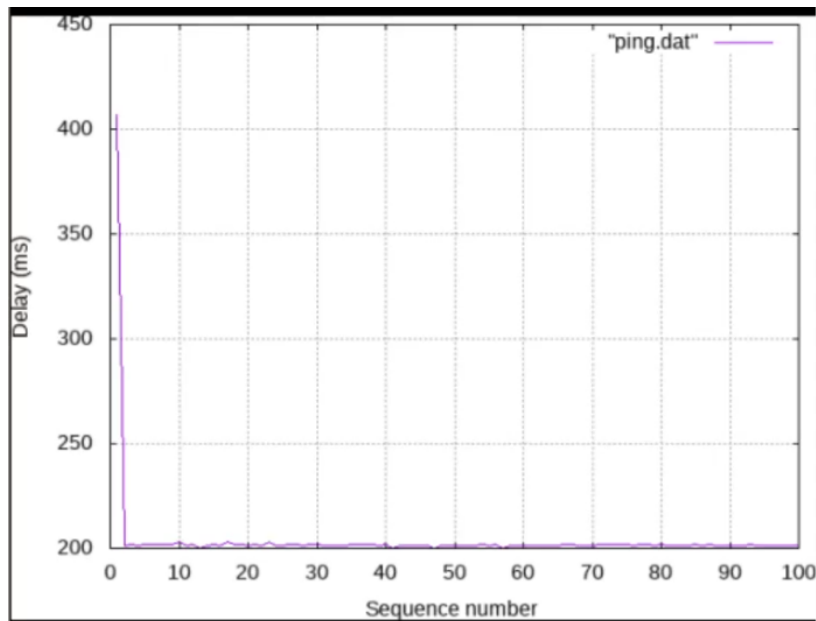


Рисунок 3.14: результат выполнения скриптов

Из файла ping.dat удалим первую строку и заново построим график. Продемонстрируем построенный в результате график (Рисунок 3.15).

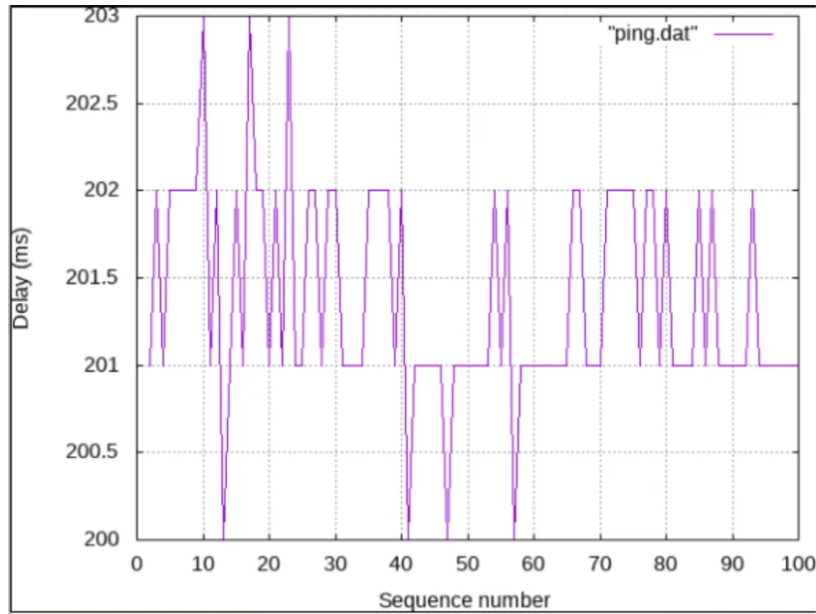


Рисунок 3.15: результат выполнения скрипта

Разработаем скрипт для вычисления на основе данных файла ping.dat минимального, среднего, максимального и стандартного отклонения времени приёма-передачи (Рисунок 3.16).

```
mininet@mininet-vm: ~/work/lab_netem_i/simple-delay
with open('ping.dat', 'r') as f:
    s = []
    for line in f.readlines():
        if '\n' in line:
            line = line.replace('\n', "")
            s.append([int(j) for j in line.split(" ")])
    s = [j[i] for j in s]
    std = (sum([(i - (sum(s) / len(s))) ** 2 for i in s]) / (len(s) - 1)) ** 0.5
    print(f"min: {min(s)} \nmax: {max(s)} \navg: {sum(s)/len(s)} \nstd: {std}")
```

Рисунок 3.16: Скрипт rtt.py

Добавим запуск скрипта в Makefile (Рисунок 3.17).

```

mininet@mininet-vm: ~/work/lab_netem_i/simple-delay
all: ping.dat ping.png

ping.dat:
    sudo python lab_netem_i.py
    sudo chown mininet:mininet ping.dat
    sudo python rtt.py

ping.png: ping.dat
    ./ping_plot

clean:
    -rm -f *.dat *.png

```

Рисунок 3.17: Makefile

Продemonстрируем работу скрипта с выводом значений на экран (Рисунок 3.18).

```

mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ make
sudo python lab_netem_i.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set delay
*** h1 : ('tc qdisc add dev h1-eth0 root netem delay 100ms',)
*** h2 : ('tc qdisc add dev h2-eth0 root netem delay 100ms',)
*** Ping
*** h1 : ('ping -c 100', '10.0.0.2', '| grep "time=" | awk \'{print $5, $7
\'s/time=//g\' -e \'s/icmp_seq=//g\' > ping.dat')
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
sudo chown mininet:mininet ping.dat
sudo python rtt.py
min: 200
max: 405
avg: 203.4
std: 20.37427571991485
./ping_plot
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$

```

Рисунок 3.18: Результат работы скрипта rtt.py

3.8 Задание для самостоятельной работы

Реализуем воспроизводимые эксперименты по изменению задержки, джиттера, значения корреляции для джиттера и задержки, распределения времени задержки в эмулируемой глобальной сети. Создадим каталоги change-delay, jitter-delay, correlation-delay, distribution-delay. Скопируем скрипты, в скрипте топологии изменим строчки в зависимости от задачи. Построим графики. Вычислим минимальное, среднее, максимальное и стандартное отклонение времени приёма-передачи для каждого случая.

- change-delay (рис. 3.19), (рис. 3.20).

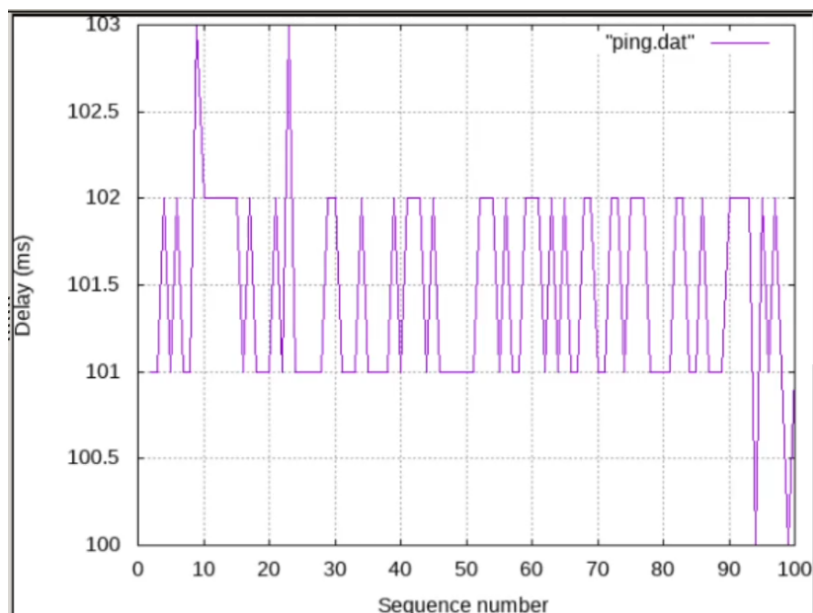


Рисунок 3.19: Просмотр графика ping.png

```

mininet@mininet-vm:~/work/lab_netem_i/change-delay$ ls
lab_netem_i.py  Makefile  ping_plot  rtt.py
mininet@mininet-vm:~/work/lab_netem_i/change-delay$ vim lab_netem_i.py
mininet@mininet-vm:~/work/lab_netem_i/change-delay$ vim Makefile
mininet@mininet-vm:~/work/lab_netem_i/change-delay$ make
sudo python lab_netem_i.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set delay
*** h1 : ('tc qdisc add dev h1-eth0 root netem delay 100ms',)
*** h2 : ('tc qdisc add dev h2-eth0 root netem delay 100ms',)
*** Change delay
*** h1 : ('tc qdisc change dev h1-eth0 root netem delay 50ms',)
*** h2 : ('tc qdisc change dev h2-eth0 root netem delay 50ms',)
*** Ping
*** h1 : ('ping -c 100', '10.0.0.2', '| grep "time=" | awk '{print $5, $7}' | sed -e
\'s/time=//g\' -e \'s/icmp_seq=//g\' > ping.dat')
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
sudo chown mininet:mininet ping.dat
sudo python rtt.py
min: 100
max: 206
avg: 102.5
std: 10.470256091723002
~/ping_plot
mininet@mininet-vm:~/work/lab_netem_i/change-delay$

```

Рисунок 3.20: Просмотр минимального, среднего, максимального и стандартного отклонения времени приёма-передачи

- jitter-delay (рис. 3.21), (рис. 3.22).

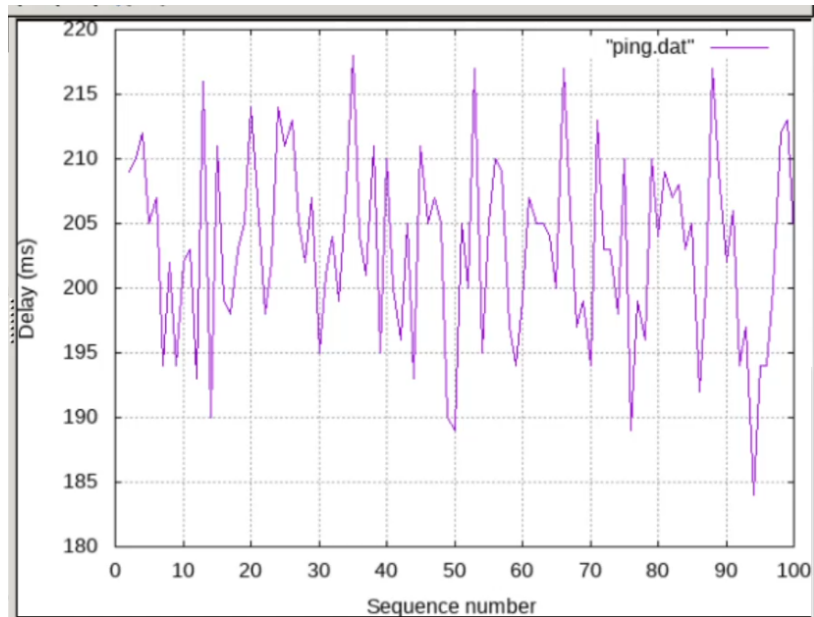


Рисунок 3.21: Просмотр графика ping.png

```
mininet@mininet-vm:~/work/lab_netem_i/jitter-delay$ make
sudo python lab_netem_i.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set delay and jitter
*** h1 : ('tc qdisc add dev h1-eth0 root netem delay 100ms 10ms',)
*** h2 : ('tc qdisc add dev h2-eth0 root netem delay 100ms 10ms',)
*** Ping
*** h1 : ('ping -c 100', '10.0.0.2', '| grep "time=" | awk \'(print $5, $7)\' | sed -e
\'s/time=//g\' -e \'s/icmp_seq=//g\' > ping.dat')
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
sudo chown mininet:mininet ping.dat
sudo python rtt.py
min: 184
max: 399
avg: 205.12
std: 20.8652628049395
./ping plot
mininet@mininet-vm:~/work/lab_netem_i/jitter-delay$
```

Рисунок 3.22: Просмотр минимального, среднего, максимального и стандартного отклонения времени приёма-передачи

- correlation-delay (рис. 3.23), (рис. 3.24).

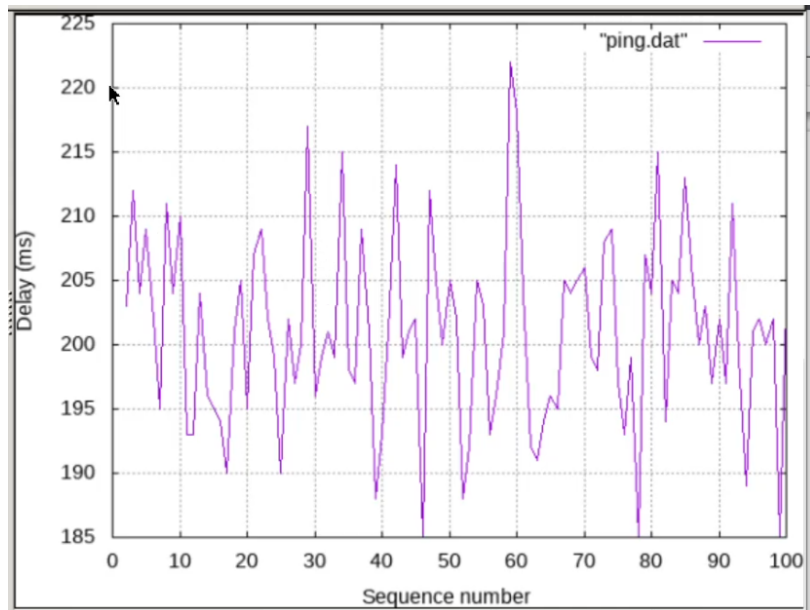


Рисунок 3.23: Просмотр графика ping.png

```
mininet@mininet-vm:~/work/lab_netem_i/correlation-delay$ make
sudo python lab_netem_i.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set delay, jitter and correlation
*** h1 : ('tc qdisc add dev h1-eth0 root netem delay 100ms 10ms 25%,)
*** h2 : ('tc qdisc add dev h2-eth0 root netem delay 100ms 10ms 25%,)
*** Ping
*** h1 : ('ping -c 100', '10.0.0.2', '| grep "time=" | awk '{print $5, $7}' | sed -
\s/time=/g\' -e \s/icmp_seq=/g\' > ping.dat')
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
sudo chown mininet:mininet ping.dat
sudo python rtt.py
min: 185
max: 417
avg: 203.14
std: 22.83538809600024
./ping_plot
```

Рисунок 3.24: Просмотр минимального, среднего, максимального и стандартного отклонения времени приёма-передачи

- distribution-delay (рис. 3.25), (рис. 3.26).

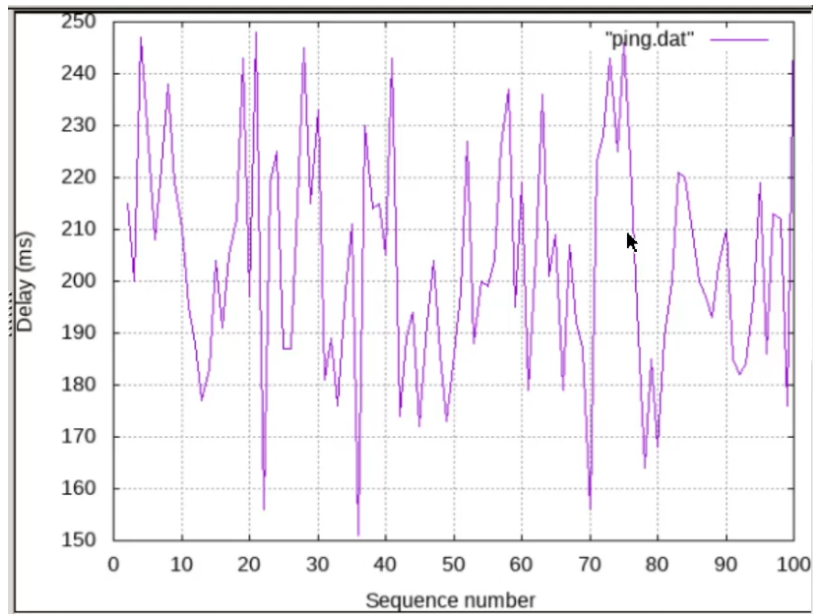


Рисунок 3.25: Просмотр графика ping.png

```
mininet@mininet-vm:~/work/lab_netem_i/distribution-delay$ make
sudo python lab_netem_i.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set delay and distribution
*** h1 : ('tc qdisc add dev h1-eth0 root netem delay 100ms 20ms distribution normal',)
*** h2 : ('tc qdisc add dev h2-eth0 root netem delay 100ms',)
*** Ping
*** h1 : ('ping -c 100', '10.0.0.2', '| grep "time=" | awk \'{print $5, $7}\'' | sed -e
\'s/time=//g\' -e \'s/icmp_seq=//g\' > ping.dat')
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
sudo chown mininet:mininet ping.dat
sudo python rtt.py
min: 151
max: 440
avg: 206.29
std: 32.42810073334109
./ping plot
mininet@mininet-vm:~/work/lab_netem_i/distribution-delay$
```

Рисунок 3.26: Просмотр минимального, среднего, максимального и стандартного отклонения времени приёма-передачи

4 Выводы

В ходе выполнения лабораторной работы я познакомилась с NETEM – инструментом для тестирования производительности приложений в виртуальной сети, а также получила навыки проведения интерактивного и воспроизводимого экспериментов по измерению задержки и её дрожания (jitter) в моделируемой сети в среде Mininet.